

Manipulator Robot Development and Analysis of Motion Planning Libraries Through ROS2 and MoveIt2



Shivam Vishwakarma , Chandan Singh , Vijay Bhaskar Semwal ,
Deepak Kumar , and M. Taufik

Abstract This study presents two key research areas: manipulator robot development and motion planning analysis. This study aims to improve robot ability in various aspects like autonomy, efficiency, and safety. These are key areas for improvement with an objective to do complex operations, navigation in dynamic environments, and interaction with humans or machines. Developing a robot model involves designing, building, and programming to achieve ergonomics. Motion planning involves information about the motion of each joint of the robot to reach our target without colliding with the obstacle that comes in the path of the robot. It requires real-time calculation and understanding of non-linear environments for safe, efficient, and robust robot navigation. In addition, robotics also possesses abundant structured resources for learning and development. One primary concern in motion planning is managing libraries that guide robots in reaching specified targets without colliding with obstacles. This study presents a comprehensive analysis on the process of developing a manipulator robot and motion planning through MoveIt2, which would manage motion planning libraries. The motion planning is carried out using a parallel motion planning setup, which results in OMPL having a 66% higher chance of solving a given configuration of a 3DOF manipulator robot when compared to a STOMP motion planning library within a 1.5-s time constraint.

S. Vishwakarma (✉) · C. Singh · D. Kumar · M. Taufik

Department of Mechanical Engineering, Maulana Azad National Institute of Technology, Bhopal, India

e-mail: shivam361v@gmail.com

C. Singh

e-mail: chandansinghrec@gmail.com

D. Kumar

e-mail: deepakkumaromre@manit.ac.in

M. Taufik

e-mail: taufikmohd86@gmail.com

V. B. Semwal

Department of Computer Science and Engineering, Maulana Azad National Institute of Technology, Bhopal, India

e-mail: vsemwal@gmail.com

Keywords Fusion360 · URDF · Motion planning · Gazebo · ROS2 · MoveIt2

1 Introduction

Robot development is a multidisciplinary approach that requires knowledge of different fields, such as mechanical, electronics, and computer science. Therefore, it is difficult for anyone to master all these areas. Hence, the robot operating system (ROS) came into the picture, simplifying robotic software development by providing a standard platform that can be utilized for any robot, irrespective of the functionality [7]. By leveraging ROS, a developer can migrate their existing project without the need to relearn about the platform, which would lead to the focus on the core features. ROS is not an operating system, it is a middleware framework that provides communication tools and plug-and-play libraries. It is also a growing community of researchers from universities, independent developers, and numerous companies, indicating market validation for the platform. It is an essential tool for managing various sensors and actuators commonly used in robots. It allows for quick prototyping and complete applications. Knowing ROS can save a very significant amount of time by creating prototype software for any mobile robot or robotics arm. Motion planning is a fundamental aspect of robotics, which guides vehicles or robots in navigating complex environments without colliding with obstacles. Over the years, motion planning algorithms have significantly developed, from collision-free paths and applying controls to handling multiple degrees of freedom robots in the nonlinear environment [12]. In 2007, Stanford University and Willow Garage Robot Company came together to overcome the inconvenience of robot trajectory planning. After a series of research and experiments, in 2010, they came up with MoveIt! function package to solve robot motion planning libraries [3]. The open motion planning library (OMPL) can provide different computing methods according to the KDL computing algorithm that could adapt motion calculation of various complex algorithms and calculate results with very high accuracy, saving a lot of computing power at robot trajectory planning [9]. Covariant Hamiltonian Optimization for Motion Planning (CHOMP) is a novel method for trajectory generation and optimization for the robotic system. This approach shares much in common with elastic band planning. A covariant gradient update rule ensures that CHOMP quickly converges to a locally optimal trajectory. A new motion planning approach that could deal with general constraints [8]. Stochastic Trajectory Optimization for Motion Planning (STOMP) uses a series of noisy trajectories. In each iteration, such trajectories are generated as well as simulated to evaluate the cost, which could be used to update candidate solution [4]. Since no gradient information is required for this process, which would lead to general constraints and additional non-smooth costs, it can be optimized [11].

2 Literature Review

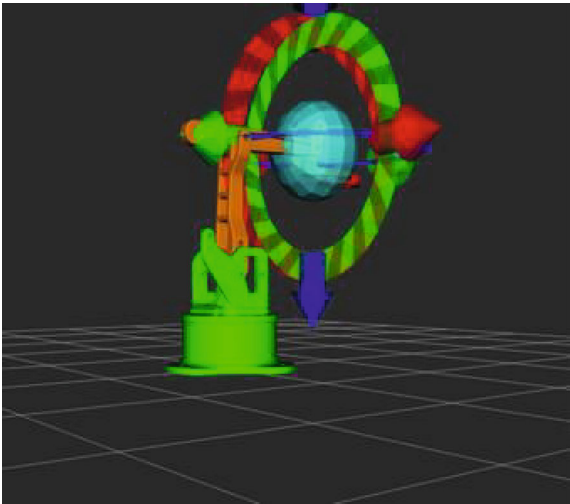
Manipulator robot development undergoes transformation, with existing advancements in recent years. A key objective is to enhance robot perception through advanced sensors, particularly vision systems. This allows robots to grasp objects effectively, even in the cluttered environment [10]. Additionally, a sensor fusion technique is developed, which allows the integration of data from multiple sensors, creating a richer understanding of their surroundings and leading to more accurate manipulation. However, learning-based methods are emerging as a new trend for high dimensions and dynamic environments. Reinforcement learning methods promise to enable robots to learn through trial and error. Researchers are also exploring a hybrid approach to combine traditional planning with learning-based methods, leveraging both strengths to achieve efficient planning while adapting to the unforeseen environment. However, robot development has significantly improved, and we are providing a structured procedure for developing manipulator robots through CAD hardware design. Thereafter, the model in the Unified Robot Description Format (URDF) is implemented to define constraints into it. In the case of motion planning libraries, we are comparing OMPL, CHOMP, and STOMP libraries, which are integrated with the MoveIt motion planning toolkit.

3 Virtual Model Development

CAD models for manipulator robots can be created using Fusion360. Researchers can analyze the robot's behavior and optimize its performance by simulating the CAD model in a nonlinear environment. The proposed method includes creating a simulation using Gazebo to validate the model and control of nonlinear systems. Additionally, the CAD design allows for the robot's dynamic behavior simulation by implementing a controller in Fig. 1.

Generally, robots are typically modeled as interconnected rigid bodies, also known as links or joints, as defined in the URDF format. This format uses extensible markup language (XML) to describe robots, including link geometry, joint types, and visual appearance. To use URDF designs, it is necessary to extract this information by identifying the links, joints, and their properties. The benefits of using URDF include standardization, modularity, scalability, and the ability to use analytical tools for robot simulations [1]. Virtual models (URDF) in robotics act as digital twins, enabling safe testing of movements, control strategies, and robot interactions in simulated environments before real-world deployment.

Fig. 1 URDF model of manipulator robot in RVIZ



4 Motion Planning Architecture with MoveIt2

A novel planning interface built into the parallel motion planning architecture simplifies the application of several motion planning algorithms to address a problem at once. This feature streamlines the planning algorithm selection process by enabling the user to select the optimal solution based on a user-defined callback [13].

Robotics mobility planning is challenging because a suitable planning algorithm to find a path in a high-dimensional space that meets specific criteria needs to be selected [2]. A good planning algorithm should be fast, generate very high-quality pathways, and cease planning after a reasonable period of time if no solution can be identified. OMPL, CHOMP, and PILZ/STOMP are just a few planning libraries that MoveIt2 currently supports. An overview of the benefits and drawbacks of each of these libraries may be found in Table 1.

Table 1 Motion planning libraries comparison

Motion planning libraries	Merit	Demerit
OMPL	Open source, versatile with many sampling-based planners, probabilistically complete (finds solution if exists)	Can get stuck in local minima, may not guarantee finding optimal solution
CHOMP	Likely domain-specific for industrial robots, potentially efficient for common tasks	May be closed-source or limited functionality outside industrial robotics
STOMP/PILZ	Effective in avoiding local minima due to stochastic nature	Not as widely applicable as OMPL, might require more tuning for specific problems

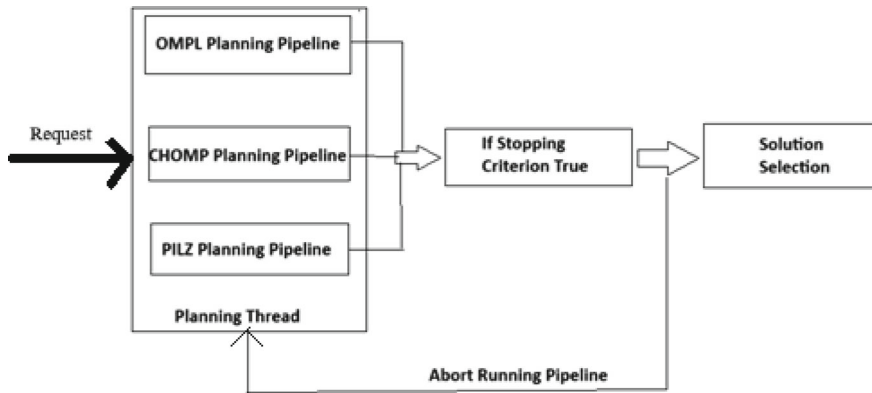


Fig. 2 Parallel motion planning setup

The movie-cpp interface is improved by the MoveIt2 parallel planning interface. Each generated `MotionPlanResponse` is gathered in thread-safe storage. When a stopping requirement is satisfied, the current `MotionPlanResponses` are fed into a system that selects the optimal choice. Numerous quality variables, such as trajectory smoothness, path length, clearance from collision objects, planning time, and execution time, can be used to identify the best solution to the planning problem [6] in Fig. 2.

Time-saving measures include stopping other pipelines if one discovers a solution and it appears likely that the best solution is among the available solutions. As an example of a heuristic for a stopping criterion, one pipeline could identify a solution and then stop the other planning pipelines. Though some planners concentrate more on feasibility, others generate excellent solutions. The user may wait for a better solution from another planner for up to a predetermined amount of time if a workable solution is found, although planning can be terminated immediately if an optimal solution is found. An illustration can be found in Fig. 3.

The parallel planning interface is advantageous. A robot arm is moved to different locations in an environment to imitate pick-and-place operations in real-world situations, and the results are compared between motion planning libraries. Figure 4 shows two resolved positional configurations, each with a combination of more straightforward and more difficult target poses. We use a 1.5-s time constraint for each query to solve the configuration 50 times for each query. The optimal option is determined by taking the shortest path possible. We have chosen OMPL, CHOMP, and STOMP as separate planning pipelines. The parallel planner selects the shortest path, which uses all three planners with the same configuration. An outcome is summarized in Table 2.

Although OMPL can solve almost all configurations of the motion planning problems, it seems to outperform the parallel planner at first glance. As a result, for every query, the parallel planning interface produces the average shortest pathways. Parallel motion planning can assist in determining the optimal solution. It can be beneficial

Fig. 3 Robot positional configuration

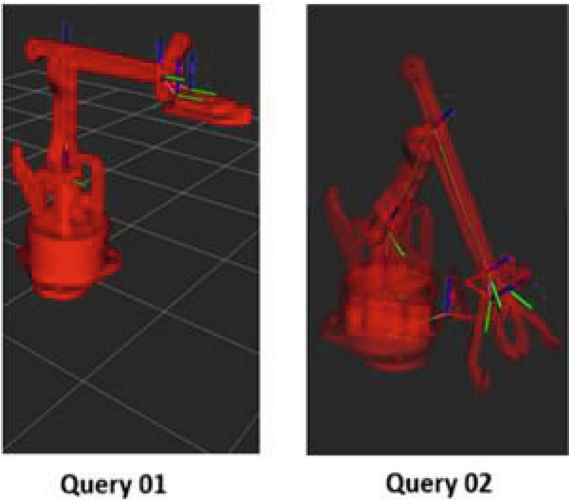


Fig. 4 Comparison of motion planning libraries

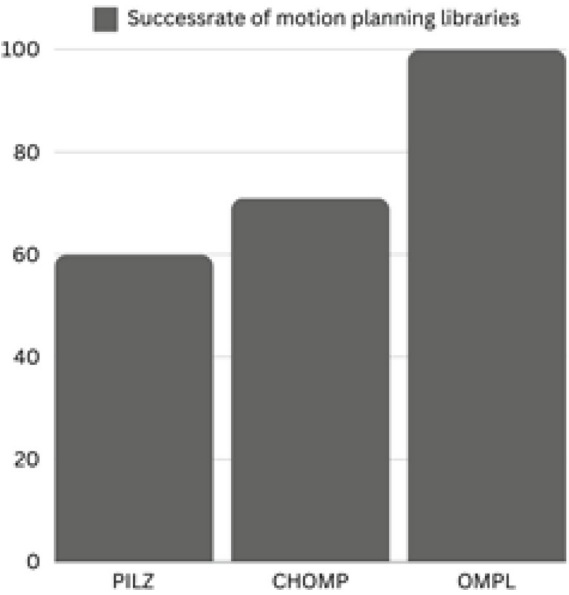


Table 2 Motion planning success rate of all queries

Planners	Percentage solved from 50 queries
OMPL	100
CHOMP	72
STOMP/PILZ	60

when you're unsure which planner will suit you the best or when you need a fall-back if your first choice doesn't work out. However, when parallel planning is used, adjusting the halting criterion and solution selection is essential to ensure optimal results. If you want to use MoveIt parallel planning interface, it is recommended that you modify these parameters according to the specific problem at hand.

5 Result

- The parallel motion planner discovered the shortest paths on average for all evaluated queries across 50 runs, each with a 1.5-s time limit. This implies that the chance of finding an outcome for a given problem is 66% higher in OMPL compared to PILZ/STOMP, as shown in Table 2.
- STOMP was only able to solve 60% of the issues in the successful cases, it found even shorter paths than OMPL. This means STOMP could be a good fit in some situations where it performs exceptionally well.

The experiment was performed for a parallel motion planning interface that utilized the CHOMP, OMPL, and PILZ/STOMP planners to replicate pick-and-place operations.

6 Conclusion

This study presents a comprehensive overview of the development process of a manipulator robot and motion planning through Moveit2, which would manage motion planning libraries. We have designed a manipulator robot from scratch, starting with CAD modeling and creating a URDF model. In addition, we have successfully applied control to the robot. Furthermore, we have analyzed a parallel motion planner that yielded results for the given configuration problem to the manipulator robot OMPL, which performed better than CHOMP and STOMP/PILZ regarding the total volume of solving the given configuration problem. The resource for my experiment contains the procedure and source code—<https://github.com/shivam361v/arduino-bot-ws.git>.

7 Future Scope

Moreover, we aim to incorporate object detection technology to enable the autonomous manipulator robot to carry out tasks without the need for direct human intervention [5]. Additionally, we seek to enhance and optimize the algorithm and controller utilized in the operation of the manipulator.

References

1. Anggraeni P, Rokhim I, Salam RM (2020) Design and development of multiple mobile manipulator robots using gazebo-ROS. In: 2020 international conference on applied science and technology (iCAST), pp 672–676
2. Chitta S, Sucan I, Cousins S (2012) Moveit! [ros topics]. *IEEE Robot Autom Mag* 19(1):18–19
3. Hernandez-Mendez S, Maldonado-Mendez C, Marin-Hernandez A, Rios-Figueroa HV, Vazquez-Leal H, Palacios-Hernandez ER (2017) Design and implementation of a robotic arm using ros and moveit! In: 2017 IEEE international autumn meeting on power, electronics and computing (ROPEC), pp 1–6
4. Kalakrishnan M, Chitta S, Theodorou E, Pastor P, Schaal S (2011) Stomp: stochastic trajectory optimization for motion planning. In: 2011 IEEE international conference on robotics and automation, pp 4569–4574
5. Mahmeen M, Sanchez RDD, Friebe M, Pech M, Haider S (2022) Collision avoidance route planning for autonomous medical devices using multiple depth cameras. *IEEE Access* 10:29903–29915
6. Pairet È, Chamzas C, Petillot Y, Kavraki LE (2021) Path planning for manipulation using experience-driven random trees. *IEEE Robot Autom Lett* 6(2):3295–3302
7. Rao D, Guodong H, Jiang Z (2020) Probplan: a framework of integrating probabilistic planning into ros. *IEEE Access* 8:106516–106530
8. Ratliff N, Zucker M, Andrew Bagnell J, Srinivasa S (2009) Chomp: gradient optimization techniques for efficient motion planning. In: 2009 IEEE international conference on robotics and automation, pp 489–494
9. Sucan IA, Moll M, Kavraki LE (2012) The open motion planning library. *IEEE Robot Autom Mag* 19(4):72–82
10. Thomas F (2014) Approaching dual quaternions from matrix algebra. *IEEE Trans Robot* 30(5):1037–1048
11. Wang B, Wang R, Song H (2023) Toward the trustworthiness of industrial robotics using differential fuzz testing. *IEEE Trans Industr Inform* 19(3):2782–2791
12. Wang T-Y, Chang C-D (2018) Hybrid fuzzy pid controller design for a mobile robot. In: 2018 IEEE international conference on applied system invention (ICASI), pp 650–653
13. Youakim D, Ridao P, Palomeras N, Spadafora F, Ribas D, Muzzupappa M (2017) Moveit!: autonomous underwater free-floating manipulation. *IEEE Robot Autom Mag* 24(3):41–51